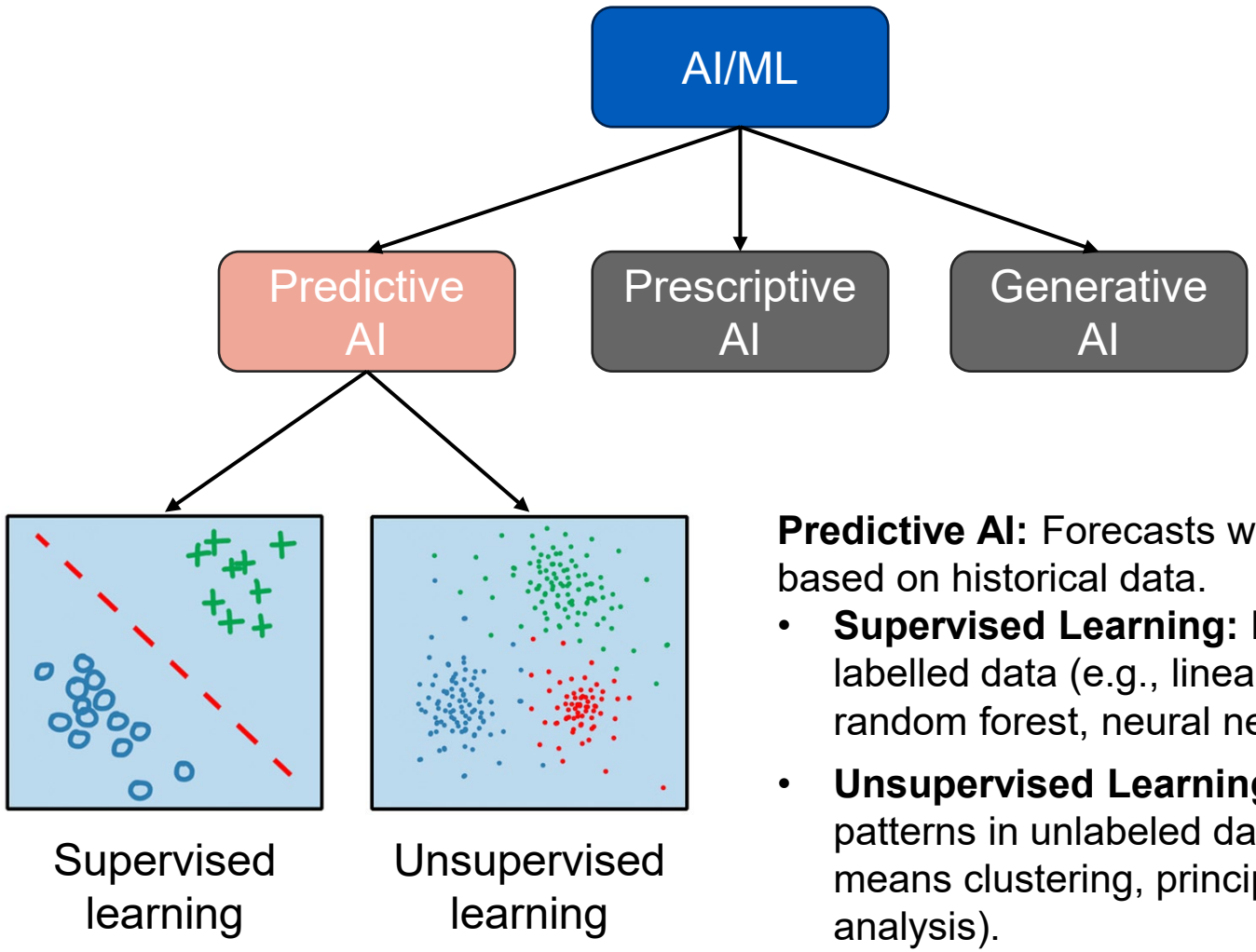
CE 400/500

Special Topics: AI for Chemical Engineers

Lecture 6: Predictive AI: Introduction to Supervised Learning, Linear Regression

University at Buffalo
The State University of New York

Predictive AI: Supervised and Unsupervised Learning



Predictive AI: Forecasts what will happen based on historical data.

- **Supervised Learning:** Learning from labelled data (e.g., linear regression, random forest, neural network).
- **Unsupervised Learning:** Discovering patterns in unlabeled data (e.g., k-means clustering, principal component analysis).

Overview of Supervised Learning

Supervised learning's tasks are well-defined and can be applied to a multitude of scenarios—like identifying spam or predicting molecular properties.

Supervised machine learning is based on the following core concepts:

- Data
- Model
- Training
- Evaluation
- Inference

https://developers.google.com/machine-learning/intro-to-ml/supervised

Data: Features and Label

Data is the driving force of ML. Data comes in the form of words and numbers stored in tables, or as the values of pixels and waveforms captured in images and audio files.

Datasets are made up of individual examples that contain features and a label. You could think of an example as analogous to a single row in a spreadsheet. Features are the values that a supervised model uses to predict the label. The label is the “answer”, or the value we want the model to predict. In a weather model that predicts rainfall amount, the data could look like:

Features

Label

date	lat	long	temp	humidity	cloud_coverage	wind_direction	atmp_pressure	rainfall
2021-09-09	49.71N	82.16W	74	20	3	N	18.6	.01
2021-09-09	32.71N	117.16W	82	42	6	SW	29.94	.23

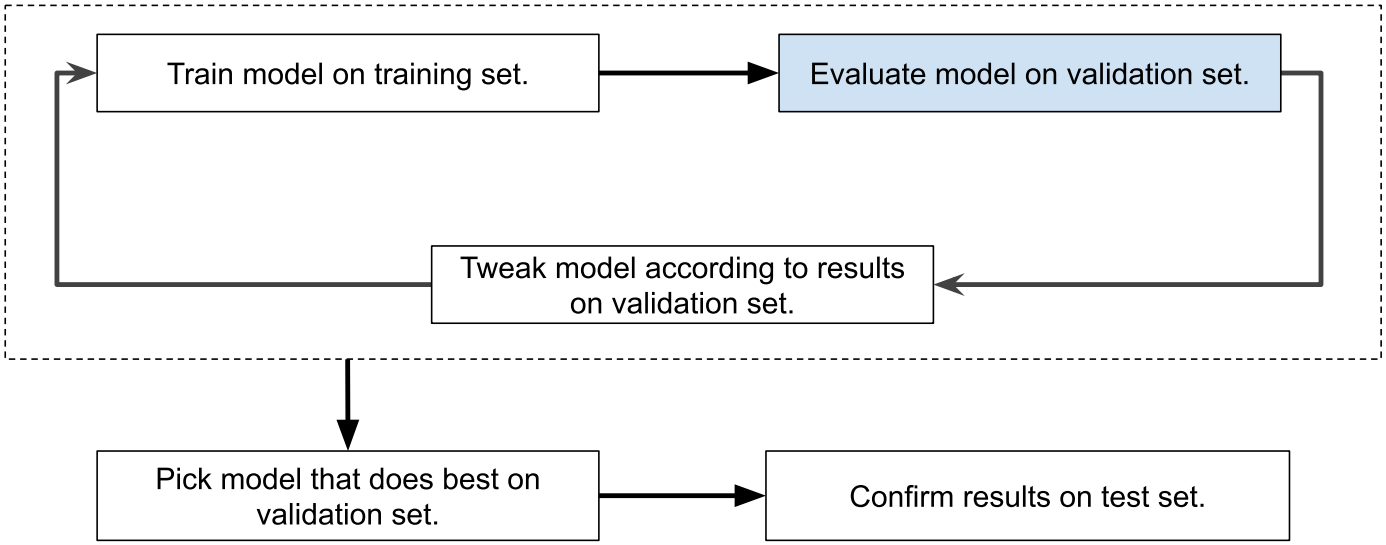
Example

https://developers.google.com/machine-learning/intro-to-ml/supervised

Data: Training, Validation, and Testing Sets

We typically split the original dataset into three subsets:

- A **training set** that the model trains on.
- A **validation set** that is used to tweak model, i.e., adjusting anything about the model —from changing hyperparameters (e.g., the learning rate), to feature selections, to completely redesigning a new model.
- A **test set** for evaluation of the trained model, which you should *never* touch before you finalize the model.

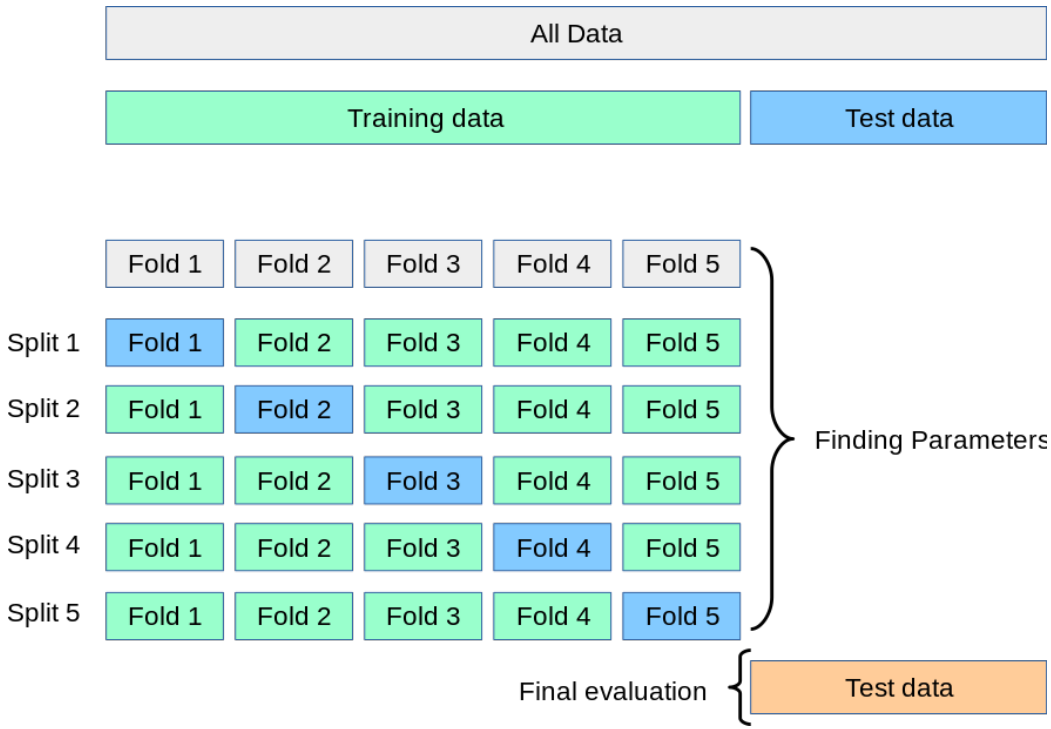


https://developers.google.com/machine-learning/crash-course/overfitting/dividing-datasets

Data: Cross-Validation

In practice, datasets are usually small, so it's inefficient to hold out a single validation set, and the results can depend on a particular random choice for the pair of (train, validation) sets.

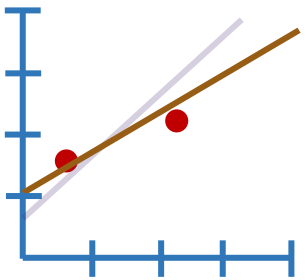
A common solution is **cross-validation (CV)**. Instead of fixing a single validation set, the training data is split into k folds. The model is trained on $k-1$ folds and validated on the remaining fold, repeating this process for all folds. The final performance is the average across folds. Though more computationally expensive, CV avoids wasting data and provides more reliable estimates, especially when datasets are small.



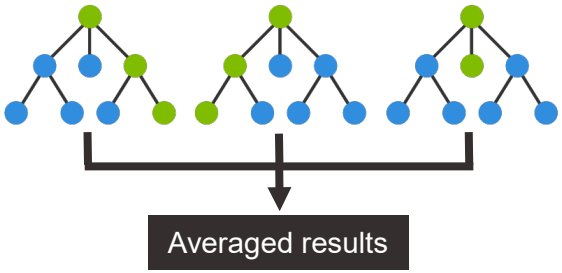
https://scikit-learn.org/stable/modules/cross_validation.html

Model: Overview

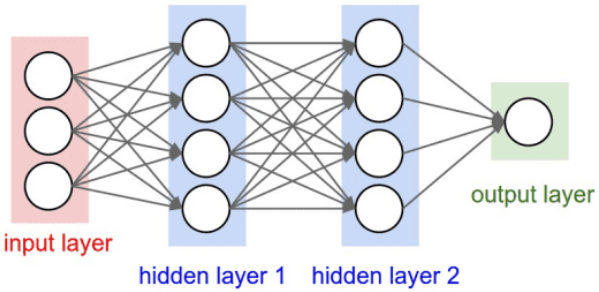
In supervised learning, **a model is the complex collection of numbers that define the mathematical relationship** from specific input feature patterns to specific output label values. The model discovers these patterns through training.



Linear regression



Random Forest
(nonlinear regression)



Multilayer Perceptron
(Feed-forward neural network, nonlinear regression)

Model: Major Supervised ML Models

Linear models:

- Linear regression, logistic regression, LASSO, etc.

Tree-based models:

- Decision trees, random forests, gradient boosting machines, extra trees, etc.

Support vector machines:

- SVM Classifier and regressor

Probabilistic models:

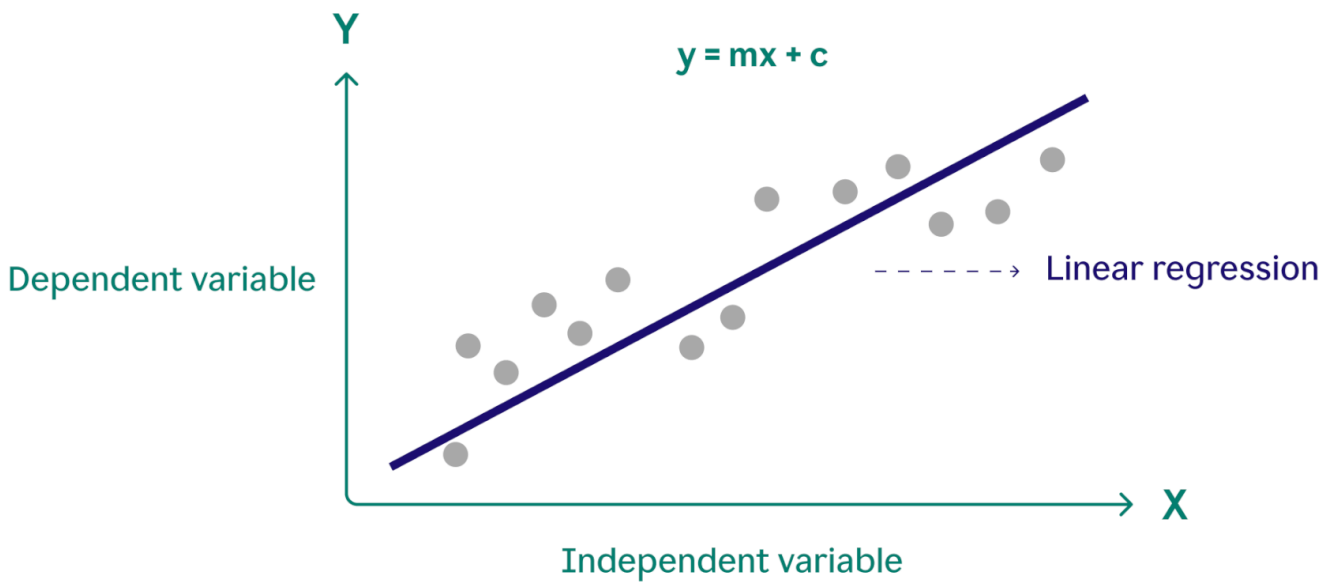
- Naïve Bayes (Gaussian, Multinomial, Bernoulli), Bayesian linear regression, etc.

Neural network models:

- Multilayer perceptron (MLP), convolutional neural networks, recurrent neural networks, transformers, etc.

Recommended reference: "Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow" by Aurélien Géron, eBook available via UB Library.

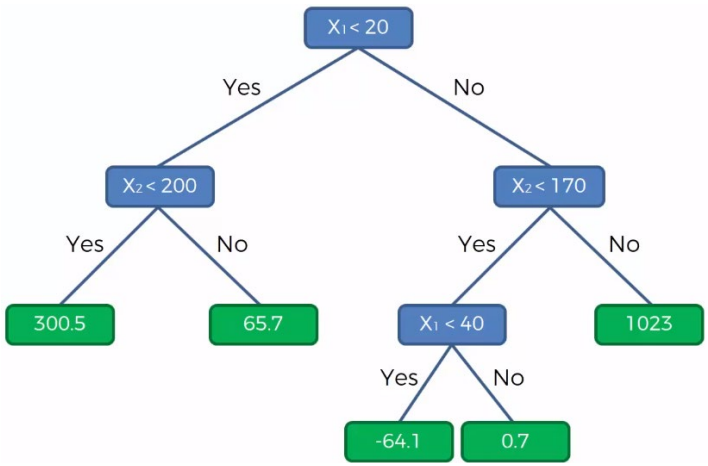
Model: Linear Models



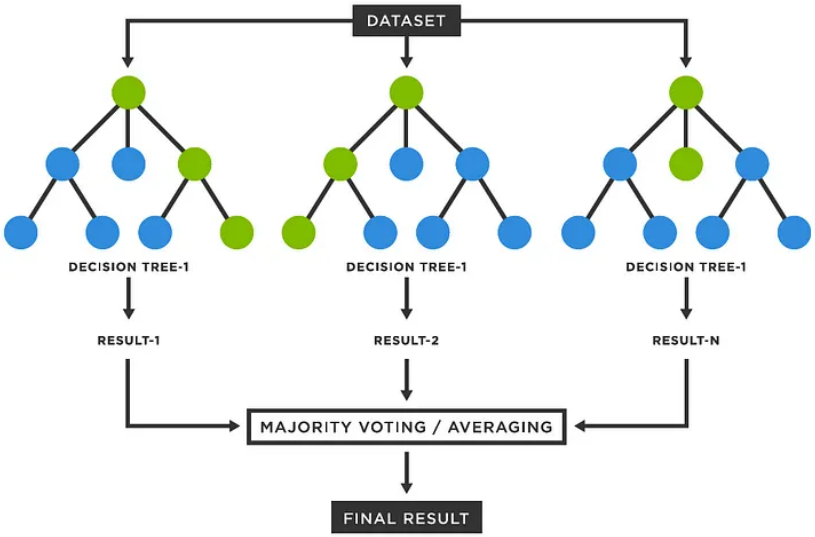
- Formally developed in the 19th century
- Linear in nature, fast estimation, and easy interpretation
- Examples include: ordinary linear regression, logistic regression, LASSO, etc.

https://www.grammarly.com/blog/ai/what-is-linear-regression/
https://www.youtube.com/watch?v=7ArmBVF2dCs&ab_channel=StatQuestwithJoshStarmer

Model: Tree-Based Models



A simple decision tree
(credit: aionlinecourse.com)

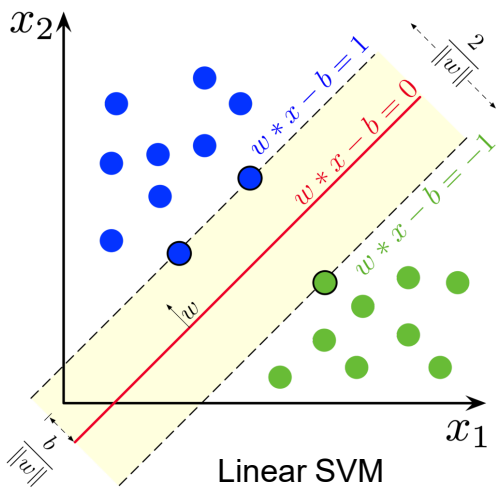


Random forest
(credit: Medium)

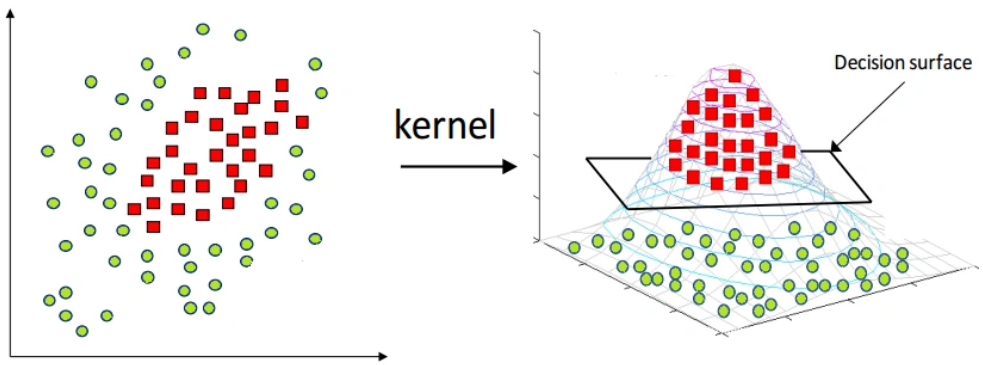
- Formalized in the 1960s—early decision tree models by Morgan & Sonquist (1963); The general method of random decision forests was first proposed by Salzberg and Heath in 1993.
- Non-linear, fast to train and interpret, handles both regression and classification tasks
- Examples include: Decision Trees, Random Forests, Gradient Boosting (e.g., AdaBoost, XGBoost)

https://www.youtube.com/watch?v=J4Wdy0Wc_xQ&ab_channel=StatQuestwithJoshStarmer

Model: Support Vector Machines (SVM)



Linear SVM
(credit: wikipedia)



The kernel trick to implicitly map low-D data to high-D data
(credit: Analytics Vidhya)

- Conceptually introduced in the 1960s by Vapnik and Chervonenkis, but popularized in the 1990s through the work of Vapnik, Cortes, and others, with the soft-margin formulation published in 1995
- A supervised learning method that finds an optimal hyperplane maximizing the margin between classes; extends efficiently to non-linear cases using the kernel trick
- Fast to train (especially linear SVMs), theoretically robust (max-margin principle), suitable for both classification and regression, and works well even with noisy data

https://www.youtube.com/watch?v=efR1C6CvhmE&ab_channel=StatQuestwithJoshStarmer
https://www.youtube.com/watch?v=wBVSbVktLIY

Model: Probabilistic Models

Likelihood of observed data y given the choice of β

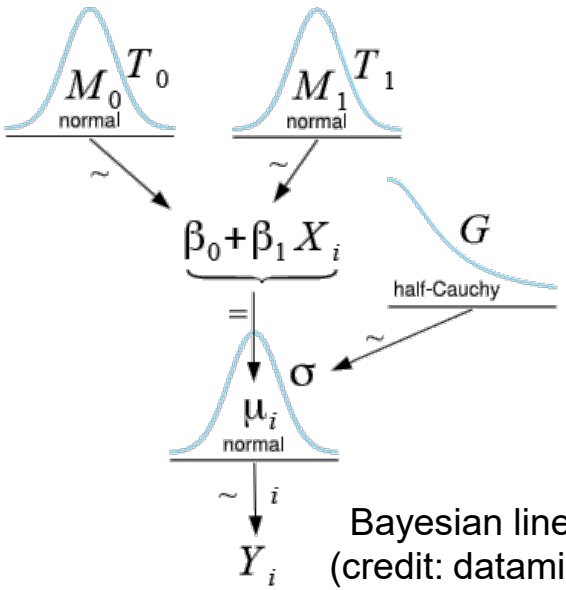
Prior belief of Parameter β before seeing the data

$$P(\beta|y) = \frac{P(y|\beta)P(\beta)}{P(y)}$$

Posterior (updated) belief about β after seeing the data

Evidence - probability of observing y for all possible β

Bayes' theorem written for the regression task

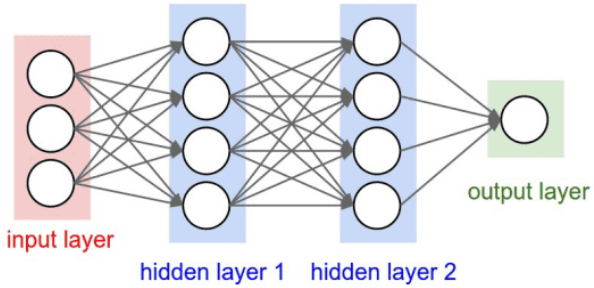


Bayesian linear regression (credit: dataminingapps.com)

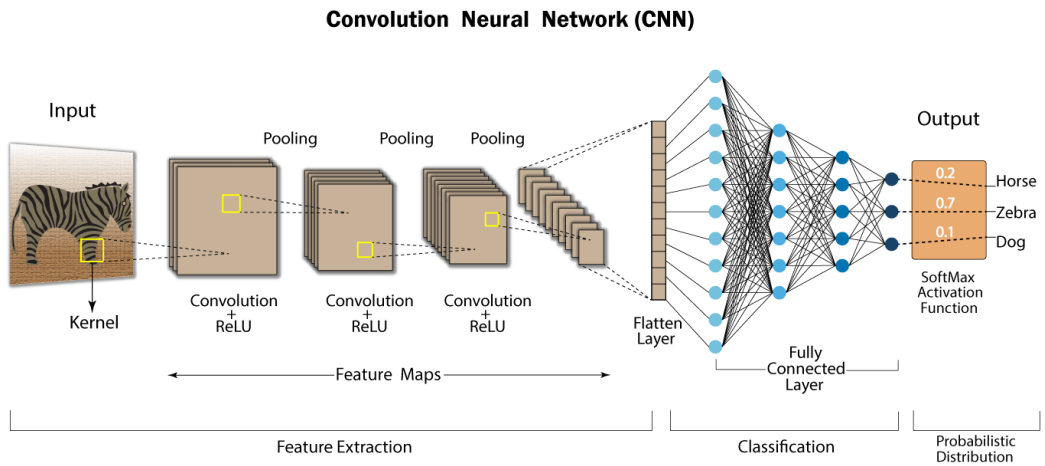
- Rooted in probability theory, formalized with Bayes' theorem (18th century)
- Models uncertainty explicitly, not just point predictions
- Often simple, computationally efficient, yet surprisingly powerful
- Examples include:
  - Naïve Bayes (Gaussian, Multinomial, Bernoulli): assumes conditional independence of features
  - Bayesian Linear Regression: parameters expressed as probability distributions

https://www.youtube.com/watch?v=9wCnvr7Xw4E&ab_channel=StatQuestwithJoshStarmer

Model: Neural Network Models



Multilayer Perceptron



- First formalized in 1943 by McCulloch and Pitts; the perceptron emerged in 1958 via Frank Rosenblatt, with deep learning later revitalized through backpropagation and deep architectures in the 1980s–2010s
- Inspired by the brain, these are layered models of interconnected “neurons” that learn patterns by adjusting connection weights
- Handles complex, nonlinear patterns; scalable via deep layers; used across vision, language, speech, and more
- Examples include: Multilayer perceptron (MLP), convolutional neural networks, recurrent neural networks, transformers, etc.

Training: Overview

Before a supervised model can make predictions, it must be trained. To train a model, we give the model a training set with labeled examples. The model's goal is to work out the best solution for predicting the labels from the features. The model finds the best solution by comparing its predicted value to the label's actual value. Based on the difference between the predicted and actual values—defined as the **loss**—the model gradually updates its solution. In other words, the model learns the mathematical relationship between the features and the label so that it can make the best predictions on unseen data.

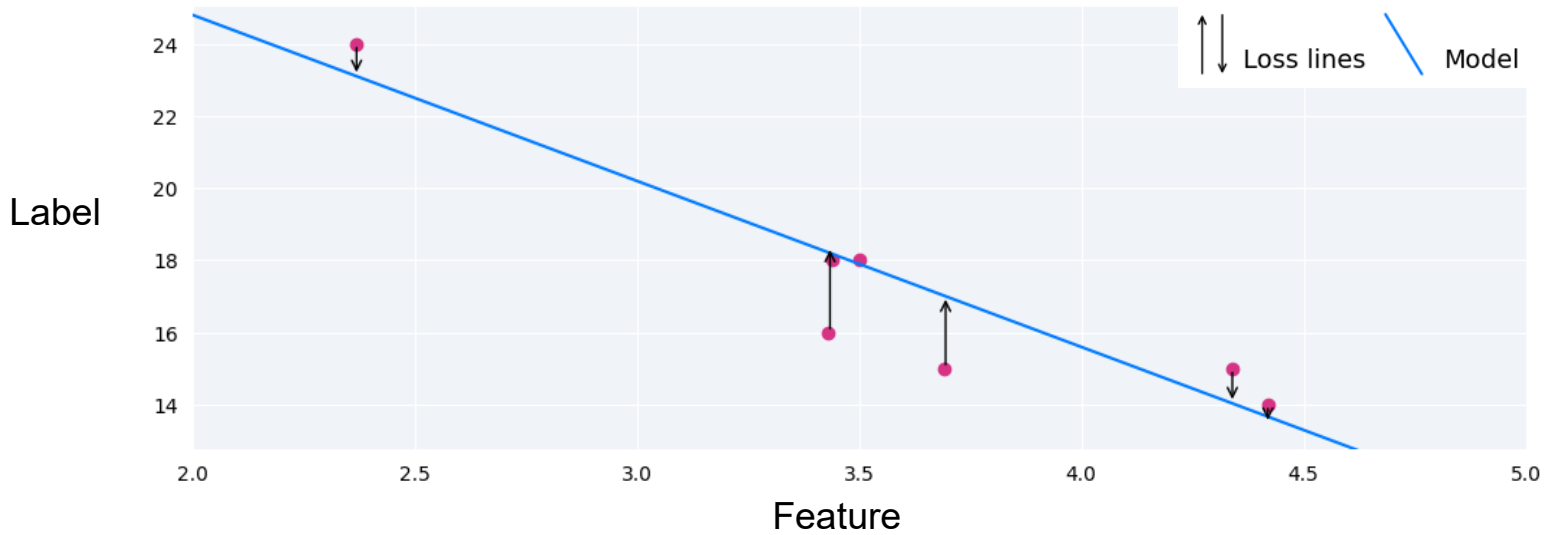
For example, if the model predicted 1.15 inches of rain, but the actual value was .75 inches, the model modifies its solution so its prediction is closer to .75 inches. After the model has looked at each example in the training set—in some cases, multiple times—it arrives at a solution that makes the best predictions, on average, for each of the examples.

https://developers.google.com/machine-learning/intro-to-ml/supervised

Training: Loss

Loss is a numerical metric that describes how wrong a model's predictions are. Loss measures the distance between the model's predictions and the actual labels. **The goal of training a model is to minimize the loss, reducing it to its lowest possible value.**

In the following image (example of linear regression), you can visualize loss as arrows drawn from the data points to the model. The arrows show how far the model's predictions are from the actual values.



https://developers.google.com/machine-learning/crash-course/linear-regression/loss

Training: Types of Loss

The loss function is one of cores of ML and it usually differentiates different ML models. You are free to design loss function, but there are four main types of loss, which are outlined in the following table.

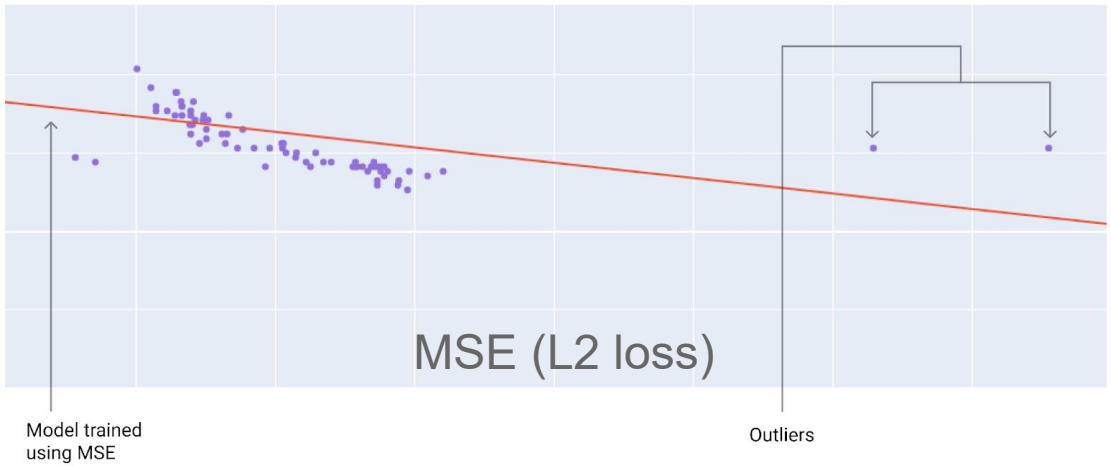
Loss type	Definition	Equation
L_1 loss	The sum of the absolute values of the difference between the predicted values and the actual values.	$\sum \lvert actual\ value - predicted\ value \rvert$
Mean absolute error (MAE)	The average of L_1 losses across a set of *N* examples.	$\frac{1}{N} \sum \lvert actual\ value - predicted\ value \rvert$
L_2 loss	The sum of the squared difference between the predicted values and the actual values.	$\sum (actual\ value - predicted\ value)^2$
Mean squared error (MSE)	The average of L_2 losses across a set of *N* examples.	$\frac{1}{N} \sum (actual\ value - predicted\ value)^2$

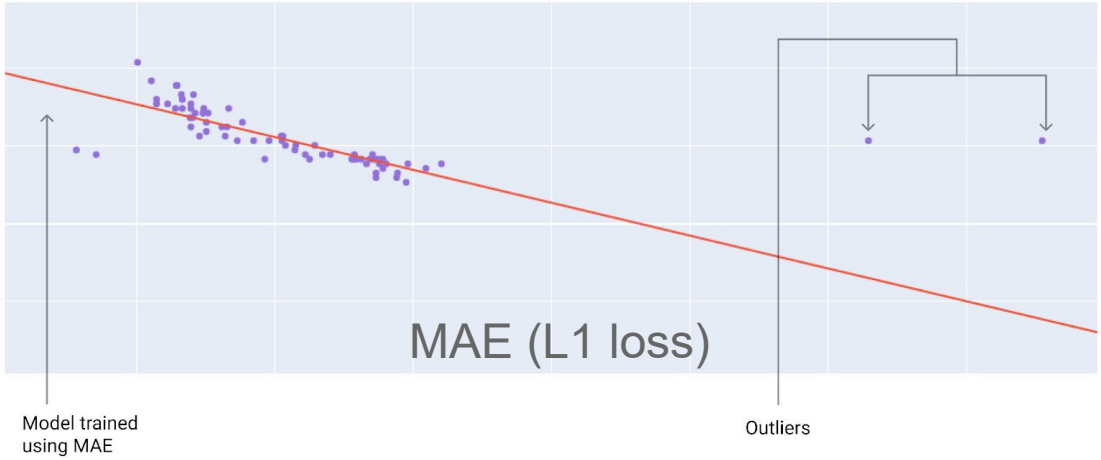
The functional difference between L_1 loss and L_2 loss (or between MAE and MSE) is squaring. When the difference between the prediction and label is large, squaring makes the loss even larger. When the difference is small (less than 1), squaring makes the loss even smaller.

Note: The name of "L_1 loss" (or "L_2 loss") is now usually considered equivalent to MAE (or MSE) in ML community, since they differ only by a constant scaling factor.

https://developers.google.com/machine-learning/crash-course/linear-regression/loss

Training: Choosing a Loss



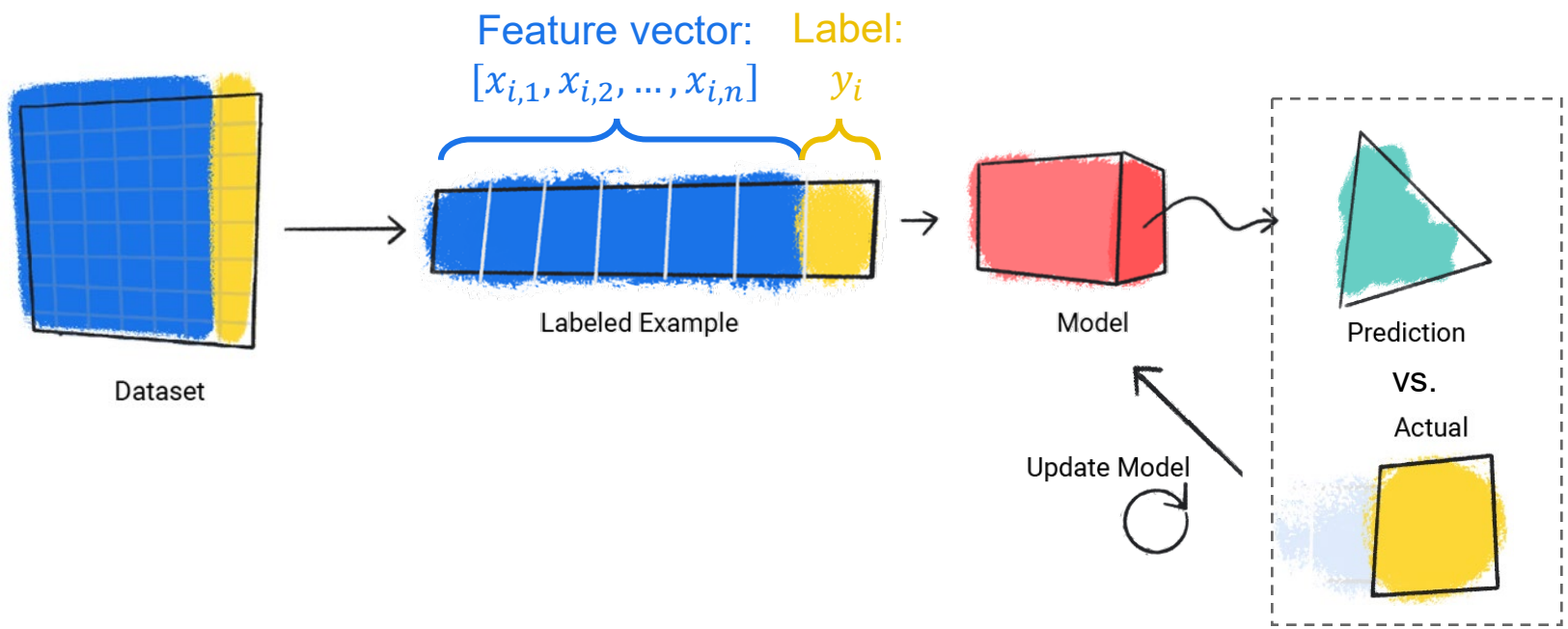


When choosing the best loss function, consider how you want the model to treat outliers. For instance,

- MSE moves the model more toward the outliers, while MAE doesn't.
- L_2 loss incurs a much higher penalty for an outlier than L_1 loss.

https://developers.google.com/machine-learning/crash-course/linear-regression/loss

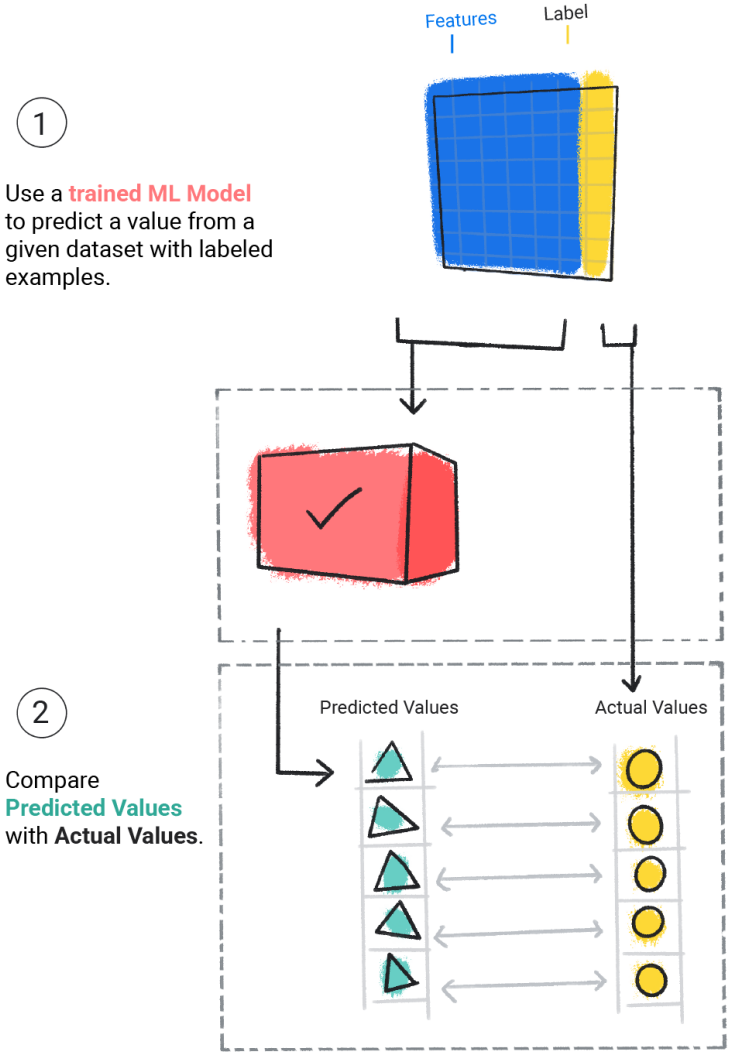
Training: Illustration of Training Process



- The model takes in a single labeled example i and provides a prediction.
- The model compares its predicted value with the actual value and updates its solution.
- The model repeats this process for each labeled example in the dataset.

https://developers.google.com/machine-learning/intro-to-ml/supervised

Evaluation



- We evaluate a trained model to determine **how well it learned**. When we evaluate a model, we use a labeled dataset, but we only give the model the dataset's features. We then compare the model's predictions to the label's true values.
- This evaluation step is generally involved in both validation (using validation set to adjust the model) and final testing (using test set to confirm the model).
- There are many evaluation metrics, such as coefficient of determination (R^2), mean absolute error (MAE), etc. We will look into these further in hands-on sessions.

https://developers.google.com/machine-learning/intro-to-ml/supervised

Inference (or Predictions)

Once we're satisfied with the results from evaluating the model, we can use the model to make predictions, called **inferences**, on unlabeled examples. In the weather app example, we would give the model the current weather conditions—like temperature, atmospheric pressure, and relative humidity—and it would predict the amount of rainfall.

The validated ML model could be deployed over:

- Data center (cloud computing)
- Local device (workstation, laptop, iPhone, iPad, etc.)

Note: Over time, model performance can degrade as real-world conditions change or new science/materials discovered, requiring monitoring and potential retraining.

Hands-on: Linear Regression for Wine Quality

See Google Colab notebook